

Três trilhas, uma ordem — e o porquê dela

TRILHA ARQ

Nasce a camada Repository (ports & adapters). O SQL sai dos services; as 2 violações plantadas caem; a régua sobe (ARQ-6).

ARQ-1 · 2 · 3 · 4 · 5 · 6

TRILHA ORM

O Prisma entra ATRÁS da mesma interface. Se a ARQ foi bem feita, o service não percebe a troca. Migrations viram lei (OP-1).

ORM-1 · 2 · 3 · 4 · 5

TRILHA AUTH

Identidade (argon2 + JWT), o guarda na porta e a matriz de permissões. Guiada até o requireAuth — depois, só contratos.

AUTH-1 ... 5 + a parte sem guia

A ordem não é estética: a tabela users da AUTH nasce por migration do Prisma (ORM), e o Prisma entra atrás das interfaces da ARQ. Pular etapa cobra juros.

Nível 1 = ARQ + ORM completas · Nível 2 = AUTH inteira (com a parte sem guia) · Nível 3 = refresh rotation OU 2º adapter + ADR

Suba o projeto — e entenda a luz vermelha

TERMINAL — 3 + 1 COMANDOS

```
npm install          # compila better-sqlite3 e argon2 (Node 22)
npm run db:reset      # cria o SQLite com o seed
npm run dev           # http://localhost:3000

npm run gate          # o juiz — rode antes de tocar em tudo
```

O que o gate confere, na ordem

- 1 **Tipos** — tsc --noEmit limpo
- 2 **Arquitetura** — dependency-cruiser — CHEGA VERMELHO
- 3 **Testes de API** — servidor real, porta efêmera
- 4 **Segredos** — gitleaks (se instalado)

A checagem 2 chega VERMELHA — de propósito

Existem 2 violações plantadas da Regra da Dependência no código. O npm run arch as acusa com arquivo e destino — encontrá-las no código é exercício do Encontro 1; corrigi-las é a segunda metade da trilha ARQ (ARQ-4 e ARQ-5).

A sua história com o gate nesta atividade é: 1 luz vermelha → todas verdes → todas verdes PARA SEMPRE. Cada tarefa que você fizer termina com o gate verde de novo — é o seu "Done when" padrão.

**Proibido "consertar" o vermelho editando gate.sh ou a régua.
Gate afrouxado = entrega reprovada, em qualquer nível.**

Duas rotas de condução — mesmo rigor, embalagens diferentes

ROTA CHAT — C·P·T·R·F·A + V.E.R.M.

Como desde o Tópico 1: **C**ontexto · **P**apel · **T**arefa · **R**estrições · **F**ormato · **A**ceitação — e nada entra no projeto sem passar pelo V.E.R.M. Você cola pedaços que consegue explicar linha a linha; o resto volta para a conversa.

Melhor para: entender antes de agir, gerar trechos pontuais, tirar dúvida de conceito no meio da trilha.

ROTA AGENTE — os 4 campos do ebook

Goal / Context / Constraints / Done when em toda tarefa (Claude Code · Codex · agy). Plan Mode obrigatório em tarefa multi-arquivo — leia o plano e EDITE antes de aprovar: é o momento mais barato do ciclo. Feche com 1 revisão adversarial por trilha e triagem justificada (aceitar 100% = ninguém julgou).

Melhor para: os movimentos repetitivos das trilhas (3 repositories, 3 reimplementações Prisma) — depois que você entendeu o primeiro.

Nas duas rotas, o "Done when" é algo que uma MÁQUINA avalia: npm run gate verde, um teste nomeado passando. "Ficar bom" não é Done when.

Regras de higiene: uma tarefa por sessão · errou 2× → /clear e tarefa reescrita · evidência no IA.md é saída de terminal colada, nunca frase.

01

Trilha ARQ — nasce a camada Repository

Extrair o port, mover o SQL para o adapter, cair as violações, subir a régua. O comportamento não muda um milímetro — os testes garantem.

O movimento de extração — três vezes o mesmo gesto

ARQ-1 — O CONTRATO QUE O SERVICE PASSA A ENXERGAR

```
// repositories/patients.repository.ts (novo)
export interface PatientsRepository { // o PORT
  findAll(): PatientJson[];
  findById(id: number): PatientRow | undefined;
  findByIdByNationalId(cns: string): unknown;
  create(input: CreatePatientInput): number;
  updatePhoto(id: number, url: string): void;
}

export class SqlitePatientsRepository // o ADAPTER
  implements PatientsRepository {
  // TODO o SQL do service vem morar aqui
}
```

O gesto, em 4 tempos

1. Escreva a interface olhando o que o service JÁ faz — o port descreve necessidade, não tecnologia.
2. Mova o SQL (e a tradução snake_case→camelCase) para o adapter SQLite.
3. O service recebe o repository e o import { db } dele desaparece.
4. Repita para Encounter (ARQ-2) e MedicationRequest (ARQ-3).

Checkpoint de pronto (cada recurso): npm run test 100% verde SEM tocar nos testes + o service sem nenhum import de src/database.

Armadilha nº 1 da trilha: se um teste quebrou, a tradução de formato ficou perdida entre as camadas. O conserto é no SEU código — editar o teste é reprovação.

A tarefa dos 4 campos, pronta para adaptar (ARQ-1)

TAREFA — COPIE, ADAPTE, USE EM PLAN MODE

Goal

Extrair a camada repository de Patient: **interface** PatientsRepository + SqlitePatientsRepository, movendo todo o SQL de @src/services/patients.service.ts para o adapter.

Context

@AGENTS.md @INVARIANTES.md @src/services/patients.service.ts
@src/repositories/LEIA-ME.md — padrão port & adapter da trilha.

Constraints

Fora de escopo: encounters, medications, rotas, controllers, public/, tests/. Sem libs novas. A tradução snake->camel vai JUNTO com o SQL para o adapter (invariante: formato não vaza).

Done when

npm run gate: checagens 1 e 3 verdes; **npm run** arch acusa UMA violação a menos que antes; nenhum diff em tests/.

Por que esta tarefa é boa

- O que fica de fora está ESCRITO — o que você exclui vale tanto quanto o que pede.
- O Done when é medível por máquina — o agente pode rodar o gate sozinho e iterar (loop fechado).
- O Context aponta arquivos, não o projeto inteiro: bancada do relojoeiro, só as peças certas.
- No plano proposto, procure: ele quis tocar algo fora do escopo? Edite ANTES de aprovar.

Rota chat equivalente: mesmo conteúdo no C-P-T-R-F-A — a Aceitação é literalmente o Done when acima.

Caçar as violações e promover a régua

A SAÍDA QUE VOCÊ VAI INVESTIGAR (npm run arch)

```
$ npm run arch
error services-nao-conhecem-a-web:
  src/services/????????.service.ts -> node_modules/express
error controllers-nao-tocam-o-banco:
  src/controllers/??????.controller.ts -> src/database.ts
```

ARQ-4 · ARQ-5

O deprecruise diz o ARQUIVO e a SETA errada; a correção é sua. Pergunta-guia: "o que esta camada precisava, e qual é o jeito de recebê-lo sem inverter a seta?"

ARQ-6 — a régua sobe

Com os repositories existindo, acrescente à régua a 4ª regra: "só repositories importam o driver". A regra nasce DEPOIS da camada — regra que sempre grita é ruído, não proteção.

É a escada do INVARIANTES.md

Advisório → teste → hook. A1 já vive na camada 2 (teste); o ARQ-6 é você ampliando a camada 2 com a fronteira nova que acabou de construir. Invariante que não é executável é decoração.

Checkpoint de pronto do NÍVEL 1 (parte ARQ): npm run gate INTEIRO verde — e a regra nova do ARQ-6 presente na régua.

02

Trilha ORM — Prisma atrás da interface

A prova material de que a ARQ valeu: trocamos o motor de persistência e o service nem fica sabendo. Roteiro detalhado em [prisma/LEIA-ME.md](#).

Introspecção primeiro, nomes depois

ORM-1 — O PRISMA CONHECE O BANCO

```
npx prisma init --datasource-provider sqlite
# .env: DATABASE_URL="file:./database/prontuario.db"
npx prisma db pull      # o Prisma Lê o banco que já existe
npx prisma generate
```

ORM-2 — DEPOIS DO SEU PASSE DE @map / @@map

```
model Patient {
  id          Int      @id @default(autoincrement())
  name        String
  birthDate   String   @map("birth_date")
  nationalId  String   @unique @map("national_id")
  active      Int      @default(1)
  @@map("patients")
}
```

Por que db pull, e não escrever do zero?

O banco JÁ existe e tem dados. A introspecção garante que o schema.prisma descreve a realidade — e te mostra, sem filtro, que o banco fala snake_case. Traduzir os nomes (passo 2) é a mesma disciplina do Tópico 2: o formato interno não vaza.

Armadilhas conhecidas

- Pulou o @map e o código encheu de birth_date → volte ao passo 2, não "conserte" no repository.
- Esqueceu npx prisma generate → o client não existe/está velho; erro de import confuso.
- active é Int no SQLite: a tradução 0/1 → boolean continua sendo SUA, no adapter.

O slide-pivô: a interface não muda

ANTES — ADAPTER better-sqlite3 (trilha ARQ)

```
export class SqlitePatientsRepository
  implements PatientsRepository {
  findByNationalId(cns: string) {
    return db.prepare(
      "SELECT id FROM patients WHERE national_id = ?"
    ).get(cns);
  }
}
```

DEPOIS — ADAPTER Prisma (ORM-3 e ORM-4)

```
export class PrismaPatientsRepository
  implements PatientsRepository { // A MESMA interface
  findByNationalId(cns: string) {
    return prisma.patient.findUnique({
      where: { nationalId: cns },
    });
  }
}
```

O service não é tocado. Nenhum teste muda. É ISSO que a arquitetura comprou — e é isso que o corretor vai procurar no seu diff.

ORM-5 — baseline de migrations

Marque o estado atual como migration inicial (conforme visto em aula). Daqui em diante vale o OP-1: esquema só muda por prisma migrate dev — inclusive a tabela users, logo ali na AUTH.

Se o migrate quiser APAGAR seu banco...

...é sintoma de baseline malfeita (migration criada sem marcá-la aplicada). Pare, respire, prisma migrate resolve --applied. Perder o seed não dói; entender por que doeria em produção é o aprendizado.

O C-P-T-R-F-A sugerido (ORM-3), pronto para adaptar

PROMPT — ROTA CHAT (C·P·T·R·F·A)

CONTEXTO: projeto Express 5 + TS em camadas; repository de Patient hoje em better-sqlite3 atrás da **interface** PatientsRepository (colo a **interface** e o adapter atual abaixo).

PAPEL: dev sênior de backend, especialista em Prisma.

TAREFA: reimplementar o adapter usando @prisma/client, mantendo EXATAMENTE a mesma **interface** e o mesmo comportamento (incluindo camelCase e active como boolean).

RESTRIÇÕES: não alterar a **interface**, o service nem os testes; não usar \$queryRaw; nomes de **model** conforme meu schema.prisma.

FORMATO: só o arquivo novo completo, comentado em português.

ACEITAÇÃO: **npm run** test 100% verde sem tocar em tests/;
npm run arch sem violação nova.

Depois da resposta: V.E.R.M.

- Verifique: roda? O gate concorda?
- Explique: cada linha do adapter, em voz alta, para a dupla.
- Reduza: a IA criou camada, cache ou opção que ninguém pediu? Corte.
- Melhore: os comentários contam o porquê, no tom do projeto?

Registro no IA.md: o prompt, a saída do teste colada, e o que você RECUSOU da resposta (recusar com motivo é senioridade).

03

Trilha AUTH — identidade e permissão

Guiada em detalhe até o `requireAuth` (AUTH-5). Depois disso, este deck entrega apenas dois contratos — e o volante é todo seu.

Os dois erros da identidade e a tabela que nasce por migration

AUTH-1 — 401 não é 403

UnauthorizedError (401): "não sei QUEM você é" — sem token, token adulterado, token expirado.

ForbiddenError (403): "sei quem é, e NÃO PODE" — o crachá é válido, a porta é que não abre. Trocar os dois é o erro clássico; os testes de ataque cobram a diferença.

Repare no nome da coluna

passwordHash — o nome já é uma promessa: senha em texto puro JAMAIS toca o banco. Quem cumpre a promessa é o service de auth (próximo slide), chamando o argon2 ANTES de qualquer INSERT.

AUTH-2 — A PRIMEIRA MIGRATION DE VERDADE (OP-1 EM AÇÃO)

```
npmx prisma migrate dev --name create-users
```

```
model User {  
  id          Int      @id @default(autoincrement())  
  name        String  
  email       String @unique  
  passwordHash String @map( "password_hash" )  
  role        String    // admin | profissional | recepcao  
  @@map( "users" )  
}
```

Checkpoint dos passos 1–2

- As duas classes novas em errors/HttpError.ts, no padrão das irmãs.
- prisma/migrations/ ganhou a pasta create-users — commitada.
- npm run gate segue inteiro verde (nada regrediu).
- Pergunta de bolso: por que role é dado, e a MATRIZ de permissões não está no banco? (pense em onde a regra precisa ser auditável)

Registro, login e o que o token carrega

AUTH-3 — SERVICE DE AUTH (VOCÊ ESCRIVE O RESTO)

```
// auth.service.ts — o coração (esboço conceitual)
const passwordHash = await argon2.hash(input.password);
// ... INSERT via repository/prisma, nunca a senha crua

// login:
const ok = user && await argon2.verify(user.passwordHash, senha);
if (!ok) throw new UnauthorizedError(
  "Credenciais inválidas." ); // NUNCA diga qual campo errou

const token = jwt.sign(
  { id: user.id, name: user.name, role: user.role },
  process.env.JWT_SECRET!,
  { expiresIn: process.env.JWT_EXPIRES_IN ?? "15m" } );
```

As 3 portas (AUTH-3)

POST /api/auth/register → 201 (senha nunca volta na resposta) · POST /api/auth/login → 200 { token, user } · GET /api/auth/me → o usuário do token (protegida).

O que vai no payload — e o que NUNCA vai

Vai: id, name, role (o front usa os três). Nunca: hash, senha, CNS, qualquer segredo — payload de JWT é LEGÍVEL por qualquer um; a assinatura só garante que não foi alterado.

AUTH-4 (schemas Zod): registro valida senha mínima; login NÃO rejeita senha curta com 400. Por quê? A resposta vai no seu README — pense no que um atacante aprende com cada mensagem diferente.

Checkpoint: A1 e A2 do requests.http respondem 201/200 — e o register repetido responde 409.

requireAuth — o guarda fica na porta, não na cozinha

AUTH-5 — O ESQUELETO COMENTADO (O CORPO É SEU)

```
// middlewares/auth.ts — o ADAPTER DE ENTRADA
export function requireAuth(req, _res, next) {
  const header = req.headers.authorization; // "Bearer x.y.z"
  // 1. sem header ou formato errado -> UnauthorizedError
  // 2. jwt.verify(token, JWT_SECRET) — ele LANÇA se o token
  //    foi adulterado ou expirou: capture e traduza p/ 401
  // 3. req.user = payload; next();
}
```

Por que middleware, e não no service?

É Hexagonal aplicada: "quem é você" é tradução de mundo externo (header HTTP) para domínio (req.user) — trabalho de borda. O service recebe o usuário RESOLVIDO e nunca lê header.

Dica de ouro: o jwt.verify sem try/catch derruba a requisição com 500 — e o ATAQUE 2 espera 401. A tradução do erro é o ponto do exercício.

Checkpoint que vale festa: aplique requireAuth num router e recarregue a página — a TELA DE LOGIN ACENDE SOZINHA. O frontend estava pronto, esperando o seu backend existir.

Ataques que já devem passar aqui: A3 (sem token → 401) e A4 (token adulterado → 401).



A partir daqui, este deck para.

De propósito. O que falta construir — e ONDE cada peça mora na arquitetura — é o coração avaliativo da atividade. Você recebe dois contratos e nenhum passo a passo:

CONTRATO 1 — a matriz de permissões

Próximo slide (e no README). Ela diz QUEM passa por cada porta — inclusive uma linha que nenhum middleware resolve sozinho. Leia-a devagar.

CONTRATO 2 — os testes de ataque

tests/auth.attacks.test.ts (eles ACORDAM sozinhos quando o login nasce) + seção AUTH do requests.http. Seu Done when do Nível 2: os 7 verdes, nenhum SKIP.

Perguntas certas para levar: o que decide papel? o que decide DOMÍNIO? e onde mora cada decisão?

A matriz de permissões — o alvo final da trilha AUTH

Ação	admin	profissional	recepcao	sem token
Ver pacientes e atendimentos	✓	✓	✓	401
Criar paciente · enviar foto	✓	✓	✓	401
Registrar atendimento	✓	✓	✗ 403	401
Ver prescrições	✓	✓	✗ 403	401
Prescrever	✗ 403	✓ só no atendimento que registrou	✗ 403	401

A última linha é diferente das outras — e é ali que mora a sua nota. Nem admin prescreve; e ser profissional NÃO basta. Que informação essa regra precisa que um middleware não tem?

Cuidado com a mão pesada: há um teste (ATAQUE 4 da suíte) que FALHA se você trancar demais — recepção PODE criar paciente.

Os sete ataques que precisam acordar verdes

setup register dos papéis funciona (201, ou 409 se repetido)

ATAQUE 1 sem token → 401 ao registrar atendimento

ATAQUE 2 token ADULTERADO (assinatura inválida) → 401

ATAQUE 3 recepção tenta prescrever → 403 (não 401!)

ATAQUE 4 recepção cria paciente → 201 (matriz sem mão pesada)

ATAQUE 5 senha errada → 401 SEM revelar qual campo errou

ATAQUE 6 profissional B não prescreve no atendimento de A → 403

O ATAQUE 6 é a última linha da matriz em forma de teste. Se a sua solução vive só em middleware, ele vai te contar.

Como rodar

```
npx tsx --test \
  tests/auth.attacks.test.ts

# enquanto o login der 404,
# eles dormem (SKIP).
# Done when: 7/7, zero SKIP.
```

O espelho manual

A seção AUTH do requests.http (A1–A7) tem os mesmos golpes para você aplicar com as próprias mãos — inclusive o A7 (token expirado, com JWT_EXPIRES_IN=5s).

Critério de pronto por nível + o que entregar

1 Nível 1 · essencial

ARQ + ORM completas. Gate inteiro verde; smoke intocado e verde; app idêntico ao primeiro dia; zero import de db fora de repositories.

2 Nível 2 · proficiente

AUTH inteira, incluindo a parte sem guia. 7/7 ataques verdes (zero SKIP); A1–A7 conferem; login do front acendeu; N1 não regrediu.

3 Nível 3 · portfólio

Refresh token com rotação OU repository em memória (2º adapter) — e uma ADR honesta registrando a decisão e o custo dela.

Entregáveis (4)

1. Repositório com commits que contam a história (marcos por trilha).
2. README atualizado: o que foi feito por nível + a resposta do AUTH-4.
3. IA.md no template novo — as DUAS rotas, com evidência colada.
4. Autoavaliação: a decisão mais difícil, um achado recusado, o que fariam diferente.

O que reprova em qualquer nível

· gate.sh ou régua editados para "passar" · teste de fumaça alterado · frontend tocado · segredo no código/commit · IA.md sem evidência real (afirmação não é evidência) · biblioteca fora do escopo fechado.

As cinco frases da condução

- 1 Refatorar sem quebrar é a habilidade — os testes são a testemunha.
- 2 A interface é o contrato: se o service percebeu a troca do banco, algo vazou.
- 3 401 é "não sei quem é"; 403 é "sei quem é, e não pode".
- 4 Papel se decide na porta (middleware); domínio se decide na cozinha (service).
- 5 Afirmação não é evidência — evidência é a saída do gate, colada no IA.md.

Bom trabalho — e lembre: o volante é seu. · Dúvidas da parte não-guiada serão respondidas com perguntas (é deliberado).